

Pseudo-SO Multiprogramado

Trabalho Prático de Sistemas Operacionais — Universidade de Brasília (UnB)

Profa. Aletéia Patrícia Favacho de Araújo

Simulador de um pseudo-Sistema Operacional multiprogramado composto por um Gerenciador de Processos, um Gerenciador de Memória, um Gerenciador de E/S e um Gerenciador de Arquivos.

Integrantes

- Lucas Licio — 211043342
- Eduardo Volpi — 190134330
- Moises Altounian — 200069306

Requisitos

- **Python 3** (testado com Python 3.10).
- Apenas a **biblioteca padrão** — nada a instalar.
- Compatível com ambiente **UNIX/Linux**.

Como executar

A partir da raiz do projeto:

```
python3 dispatcher.py <processes.txt> <files.txt> <string.txt>
```

Usando os arquivos de exemplo já incluídos na pasta `entradas/`:

```
python3 dispatcher.py entradas/processes.txt entradas/files.txt entradas/string.txt
```

Estrutura do projeto

<code>processo.py</code>	Classe Processo (PCB): dados e estado de cada processo.
<code>leitor.py</code>	Leitura e parsing dos três arquivos <code>.txt</code> de entrada.
<code>memoria.py</code>	Gerência de memória: paginação, frames e LRU local.
<code>filas.py</code>	Filas de prontos: tempo real (FIFO) e usuário (MLFQ).
<code>recursos.py</code>	Gerência de E/S: semáforos e alocação exclusiva.
<code>arquivos.py</code>	Sistema de arquivos: alocação contígua e first-fit.
<code>dispatcher.py</code>	Processo despachante: integra tudo e imprime a saída.
<code>entradas/</code>	Arquivos de exemplo (<code>processes.txt</code> , <code>files.txt</code> , <code>string.txt</code>).

Formato dos arquivos de entrada

O programa lê **três** arquivos `.txt`.

1. Processos — uma linha por processo (PID atribuído de 0 a N-1 na ordem):

```
<t_inicializacao>, <prioridade>, <t_processador>, <working_set>, <impressora>, <scanner>, <modem>, <sata>
```

Prioridade `0` = processo de tempo real; `1`, `2` ou `3` = processo de usuário.

As últimas quatro colunas são flags (0 ou 1) de requisição de recurso de E/S.

2. Arquivos — configuração do disco e operações do sistema de arquivos:

```
Linha 1:          quantidade total de blocos do disco
Linha 2:          quantidade de segmentos já ocupados (n)
n linhas seguintes: <nome>, <primeiro_bloco>, <quantidade_blocos>
demais linhas:    <pid>, <codigo>, <nome>[, <blocos>]
```

Código da operação: `0` = criar (exige a quantidade de blocos); `1` = deletar.

3. Strings de referência — uma linha por processo, com a sequência de páginas acessadas durante a execução:

```
<pagina_1>, <pagina_2>, ..., <pagina_n>
```

A linha *i* corresponde ao processo de PID *i*.

Saída

O despachante imprime, na ordem:

1. Para cada processo escalonado, o bloco `dispatcher =>` (PID, frames, prioridade, tempo e recursos) e as mensagens de execução do processo.
2. O resultado de cada operação do sistema de arquivos (``Operação N => Sucesso/Falha``).
3. O mapa de ocupação do disco (`0` indica bloco vazio).
4. O número de faltas de página de cada processo.

Observações

- Foram usados apenas recursos padrão da linguagem (`sys` e `collections.deque`), conforme exige a especificação.
- Os módulos foram escritos de forma independente e podem ser testados isoladamente (por exemplo, `memoria.simular_lru` e `arquivos.SistemaArquivos`).